

멀티 시뮬레이터 백엔드 설계안 (mock / Gazebo / Isaac Sim)

상태: 설계안 (코드 미반영)

대상 환경: ROS 2 Humble, MoveIt2, ros2_control

목적: 현재 RViz2 기반 mock 동작을 유지하면서, Gazebo(Fortress) 및 NVIDIA Isaac Sim 을 교체 가능한 백엔드로 갖을 수 있는 구조를 정의한다.

1. 목표와 비목표

목표

- rdfp 애플리케이션 계층(MoveIt 클라이언트 / recorder / session / dataset)을 시뮬레이터에 무관하게 단 한 줄도 바꾸지 않고 유지한다.
- 시뮬레이터를 `backend:=mock|gazebo|isaac` 같은 단일 스위치로 교체한다.
- 새 시뮬레이터 추가 비용을 "backend launch 파일 1개 + URDF 하드웨어 분기 1개" 수준으로 낮춘다.
- 현재 `rdfp_panda_mock / replay_panda_mock` 의 복불 중복(argument 선언, YAML 로딩, controller 기동 체인)을 제거한다.

비목표

- 이 문서에서 실제 Gazebo / Isaac 연동 코드를 구현하지 않는다 (구조만 정의).
- 물리 파라미터 튜닝(마찰, PID gain, 접촉 모델)은 백엔드별 후속 과제로 둔다.
- dataset / rosbag / replay 파이프라인 내부 로직은 변경 대상이 아니다(토픽 계약만 유지되면 그대로 동작).

2. 현재 구조 분석 (왜 이 설계가 필요한가)

2.1 RViz2 는 시뮬레이터가 아니다

현재 "panda mock" 동작의 실체는 RViz2 가 아니라 ros2_control 의 `mock_components` (무물리 fake hardware) 이다. RViz2 는 MoveIt 플래닝 결과를 보여주는 시각화 도구일 뿐이며, Gazebo / Isaac 도입 후에도 그대로 띄워둘 수 있다. 즉 교체 대상은 "RViz → 시뮬레이터"가 아니라 "하드웨어 인터페이스 백엔드" 이다.

`ros2_control_hardware_type` 은 이미 launch argument 로 노출되어 있고 ([launch/launch_helper.py](#) 의

`build_moveit_config()`), URDF xacro 의 `<ros2_control><hardware>` 플러그인을 주입한다. 올바른 추상화 경계가 이미 코드에 존재한다는 뜻이다.

2.2 현재 launch 파일의 책임 혼재

[panda_mock.launch.py](#),

[rdfp_panda_mock.launch.py](#),

[replay_panda_mock.launch.py](#)

는 각각 다음을 한 파일에서 수행한다.

- 1. 로봇 백엔드: static_tf, robot_state_publisher, ros2_control_node, 3개 controller spawner 의 순차 기동 체인.
- 2. MoveIt: move_group, servo.
- 3. 앱: camera, ee_pose, gripper, session_control, image_recorder, image_viewer, target_joint_states_*, rviz.
- 4. 설정: YAML 로딩 + _declare_arguments().

→ (1) 만 시뮬레이터마다 달라지는데, (2)(3)(4) 가 같은 파일에 묶여 있어 시뮬레이터를 추가하면 전체가 복제된다. 실제로 rdfp_panda_mock 과 replay_panda_mock 의 _declare_arguments() 는 약 130줄이 완전히 동일하다.

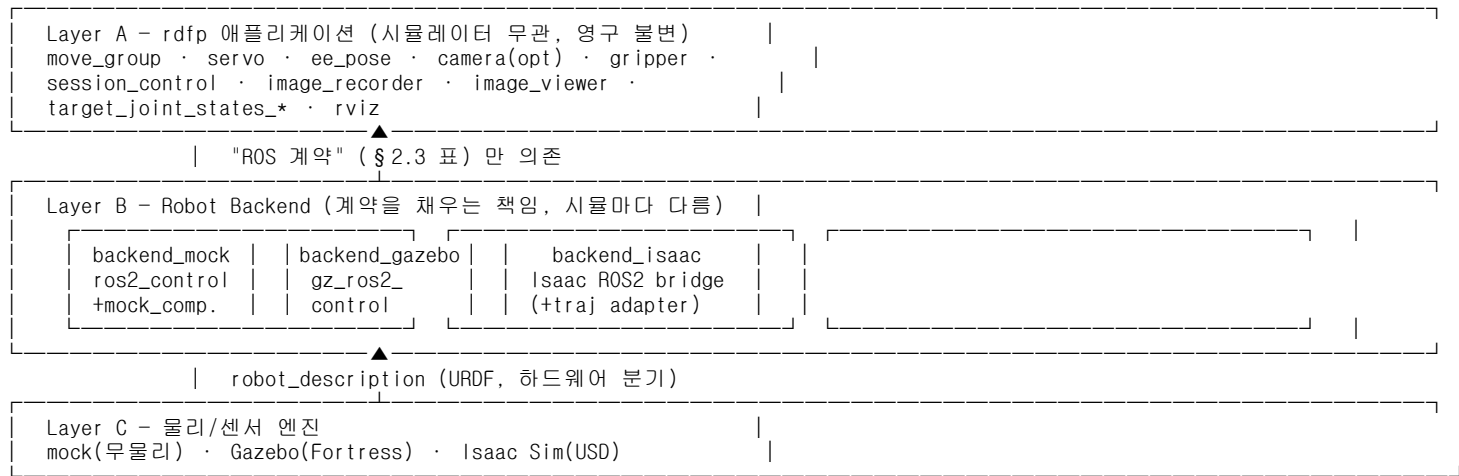
2.3 rdfp 가 실제로 의존하는 "계약"

앱 계층은 물리엔진을 직접 모른다. 다음 ROS 인터페이스에만 의존한다.

계약 항목	메시지/타입	방향(앱 관점)	제공 주체
/joint_states	sensor_msgs/JointState	구독	백엔드
/panda_arm_controller/... (FollowJointTrajectory)	control_msgs action + joint_trajectory 토픽	명령	백엔드(controller)
/panda_hand_controller/... (gripper)	control_msgs action	명령	백엔드(controller)
TF: world→panda_link0→...→panda_hand	tf2	구독	robot_state_publisher + 백엔드
robot_description	URDF 문자열	공유	공통
/camera/image_raw, /camera/camera_info	sensor_msgs/Image, CameraInfo	구독	카메라(웹캠 or 시물 센서)
/session	rdfp_msgs/SessionCommand	내부	rdfp (시물 무관)

이 계약을 어떤 백엔드든 동일하게 만족시키면 앱은 불변이다. 이것이 설계의 복극성이다.

3. 목표 아키텍처: 3계층 분리



핵심 원칙:

- Layer A 와 B 를 서로 다른 launch 파일로 물리적으로 분리한다.

- 최상위 bringup launch 가 backend:=... 값에 따라 B 중 하나를 include 하고, 그 위에 항상 동일한 A 를 include 한다.
- B 의 모든 구현체는 동일한 "백엔드 계약" (§ 5)을 만족해야 한다.

4. 제안 파일/디렉터리 구조

기존 helper 모듈은 재사용하고, launch 디렉터리를 역할별로 재배치한다.

```
src/rdfp/launch/
├── bringup/
│   └── rdfp_bringup.launch.py      # ★ 최상위 진입점. backend:=mock|gazebo|isaac
│                                   #   + mode:=teleop|replay 로 A/B 조합
├── app/
│   └── rdfp_app.launch.py          # ★ Layer A 전체 (시울 무관). 항상 동일.
│                                   #   mode 인자로 teleop/replay 노드셋 분기
├── backends/
│   ├── backend_mock.launch.py      # ★ 현재 ros2_control + mock_components
│   ├── backend_gazebo.launch.py    # ★ (신규) Gazebo Fortress + gz_ros2_control
│   └── backend_isaac.launch.py      # ★ (신규) Isaac Sim bridge + traj adapter
├── helpers/
│   ├── launch_helper.py            # 기존 *_launch_helper.py 이동(또는 유지)
│   ├── controller_launch_helper.py # build_moveit_config 등 - § 7 대로 일반화
│   ├── controller_startup_launch_helper.py
│   ├── camera_launch_helper.py
│   ├── ee_pose_launch_helper.py
│   ├── gripper_launch_helper.py
│   └── config_args_helper.py        # ★ (신규) _declare_arguments / YAML 로딩 단일화
└── (기존 panda_mock.launch.py 등은 thin 호환 래퍼로 남기거나 단계적 폐기)
```

주의: ROS2 launch 러너가 단일 스크립트로 로드하므로, sibling import 를 위한

sys.path.insert(0, os.path.dirname(__file__)) 패턴은 디렉터리 분리 후에도

각 launch 파일에서 helpers/ 경로를 추가하는 형태로 유지해야 한다. [setup.py](#).

data_files 의 launch/*.py glob 도 하위 디렉터리를 포함하도록

launch/**/*.py (또는 디렉터리별 명시 항목)로 확장해야 한다.

4.1 사용 예시 (목표 UX)

```
# 현재와 동일 (기본 backend=mock)
ros2 launch rdfp rdfp_bringup.launch.py

# Gazebo 백엔드로 동일 앱 실행
ros2 launch rdfp rdfp_bringup.launch.py backend:=gazebo

# Isaac Sim 백엔드 (이미 떠 있는 Isaac 에 연결)
ros2 launch rdfp rdfp_bringup.launch.py backend:=isaac

# replay 모드 (어느 백엔드든 조합 가능)
ros2 launch rdfp rdfp_bringup.launch.py backend:=mock mode:=replay
```

5. 백엔드 계약 (Backend Contract)

backends/backend_*.launch.py 는 모두 다음을 보장해야 한다. 이것이 Layer A 와의 인터페이스다.

5.1 반드시 제공(MUST publish/serve)

- /joint_states (sensor_msgs/JointState) — arm 7 + finger 2 관절, 이름은 URDF 와 일치 (panda_joint1..7, panda_finger_joint1..2).
- panda_arm_controller 의 FollowJointTrajectory 액션 및 /panda_arm_controller/joint_trajectory 토픽.
- panda_hand_controller 의 그리퍼 액션.

- `robot_description` 파라미터 (`robot_state_publisher` 가 발행하는 TF 의 소스).
- TF 트리: `world` → `panda_link0` → ... → `panda_hand`(+ `panda*finger`).

5.2 보장 조건(MUST hold)

- `controller` 이름·관절 이름·액션 인터페이스가 **백엔드 무관하게 동일**해야 한다 (MoveIt 의 `moveit_controllers.yaml` 이 그대로 재사용되도록).
- 백엔드 기동이 끝났음을 알리는 **신호**를 제공한다. 현재는 `panda_hand_controller` `spawner` 의 `OnProcessExit` 가 그 역할을 하는데 (`controller_startup_launch_helper.py`), Isaac 처럼 `spawner` 가 없는 경우 대체 신호(예: `controller` 활성 확인 노드)가 필요하다 → § 6.3.

5.3 선택(MAY)

- `/camera/image_raw`, `/camera/camera_info` — 시물 센서로 제공하면 OpenCV `camera_node` 를 끈다(`enable_camera_node:=false`). 제공하지 않으면 기존 웹캠 `camera_node` 가 그대로 채운다.
- `world` 정의 / 환경 오브젝트.

5.4 카메라 토픽 일원화 (중요)

`recorder` / `dataset` / `image_viewer` 는 `camera_image_topic` 등으로 remap 되는 **토픽 이름**에만 의존한다. 따라서 백엔드가 시물 카메라를 제공하든 웹캠을 쓰든, 최종 토픽 이름(`/camera/image_raw`)만 같으면 녹화·데이터셋 파이프라인은 무변경이다. 백엔드는 카메라 소스만 바꾸고 토픽 이름은 계약으로 고정한다.

6. 백엔드별 구현 개요

6.1 backend_mock (현행 유지)

- 내용: `ros2_control_node(controller_manager)` + `joint_state_broadcaster` / `panda_arm_controller` / `panda_hand_controller` `spawner` 순차 기동 + `static_tf` + `robot_state_publisher`.
- 출처: 현재 `panda_mock.launch.py` 의 `ros2_control` 부분을 그대로 추출.
- URDF 하드웨어: `mock_components/GenericSystem`.
- 변경 위험: 낮음 (기존 코드 이동).

6.2 backend_gazebo (신규, Fortress 기준)

- Gazebo Fortress 서버/클라이언트 기동 (`ros_gz_sim`).
- 로봇 스폰: `ros_gz_sim create -topic robot_description` (`robot_state_publisher` 가 발행하는 `description` 사용).
- `controller_manager` 는 **별도 노드로 띄우지 않는다** — `gz_ros2_control` 플러그인이 Gazebo 프로세스 내부에서 호스팅한다. 따라서 `backend_gazebo` 는 `ros2_control_node` 를 포함하지 않고, `controller spawner` 만 스폰 완료 이벤트 이후에 실행한다.
- URDF 하드웨어: `gz_ros2_control/GazeboSimSystem` + `<gazebo>` 플러그인 블록. 추가로 모든 링크에 `<inertial>`/`<collision>` 필요 (§ 7).
- 카메라: Gazebo 카메라 센서 플러그인 → `ros_gz_image/ros_gz_bridge` 로 `/camera/image_raw` 브리지. `enable_camera_node:=false` 로 웹캠 노드 끄.

- 클럭: Gazebo `/clock` → `use_sim_time:=true` 를 앱 포함 전체 노드에 전파해야 한다 (§ 8.3).

6.3 backend_isaac (신규, 연결형)

- Isaac Sim 은 보통 별도 GPU 호스트/컨테이너에서 이미 실행 중이고, ROS2 `bridge(isaacsim.ros2.bridge)`로 연결된다. 따라서 `backend_isaac` 는 Gazebo 처럼 시뮬 프로세스를 띄우지 않고 **브리지/어댑터/remap** 만 올리는 **얇은 launch** 가 된다.
- 두 가지 연동 방식 중 택일(또는 단계적):
 - (A) **ros2_control 하드웨어 플러그인** (`isaac_ros2_control`): Gazebo 와 동일하게 `controller_manager` 를 Isaac 측이 호스팅. 계약 충족이 가장 깔끔.
 - (B) **토픽 브리지** (OmniGraph Action Graph): Isaac 이 `/joint_states` 발행 + `JointState/JointTrajectory` 명령 구독. 이 경우 MoveIt 의 `FollowJointTrajectory` 액션을 Isaac 명령으로 변환하는 **trajectory adapter** 노드가 `backend_isaac` 에 추가로 필요하다(기존 `target_joint_states_executor` 와 유사한 역할의 일반화 버전).
- 기동 신호 (§ 5.2): `spawner` 가 없으므로, "controller/토픽이 활성화되었는지"를 확인하고 Layer A 기동을 트리거하는 **readiness gate** 노드가 필요하다.
- URDF↔USD 정합성: 진실원본이 USD 씬이므로, 관절 이름/순서/프레임이 URDF 와 일치해야 한다. 이것이 Isaac 백엔드의 핵심 계약 항목이다.
- 카메라: Isaac 의 강점인 고품질 RGB/Depth 센서 → `/camera/image_raw` 로 발행, 웹캠 노드 끄.

기동 신호 일반화: 현재 `create_controller_startup_handlers()` 는 `panda_hand_controller` `spawner` 의 종료에 강결합되어 있다. 이를 "백엔드가 제공하는 `ready` 이벤트" 추상으로 바꾸면 `mock/gazebo(spawner 기반)`와 `isaac(readiness gate 기반)`을 동일 인터페이스로 다룰 수 있다. → § 9 Phase 3.

7. URDF / xacro 전략

7.1 하드웨어 분기 일원화

`ros2_control_hardware_type` ∈ {`mock`, `gazebo`, `isaac`} **단일 인자로** `<ros2_control><hardware>` 플러그인과, Gazebo 의 경우 `<gazebo>` 시스템 플러그인 블록까지 조건부 포함한다.

문제: 현재 사용하는 `panda.urdf.xacro` 는 `rdp` 가 아니라 외부 `moveit_resources_panda_moveit_config` 패키지 소유다. 외부 패키지를 수정하면 재현성이 깨진다. 권장:

- **rdp 패키지 안에 description 을 포크한다.** 예: `src/rdp/description/panda/panda.urdf.xacro` (+ `ros2_control` 매크로). `build_moveit_config()` 의 `.robot_description(file_path=...)` 가 이 포크를 가리키도록 변경 (§ 7.3).
- 포크된 `xacro` 에서 `ros2_control_hardware_type` 값에 따라 `include` 할 하드웨어 매크로를 분기.

7.2 물리 속성 보강

mock 은 <inertial>/<collision>/마찰이 없어도 동작하지만, Gazebo·Isaac 은 필수다. 지금부터 **description** 에 제대로 채워두면 두 시물 모두 재사용된다.

- 각 링크 <inertial> (mass, inertia tensor)
- <collision> 지오메트리 (시각 메시보다 단순화 권장)
- joint <dynamics> (damping/friction), Gazebo <gazebo> 마찰 계수

7.3 build_moveit_config 일반화

[launch_helper.py](#) 의

build_moveit_config() 는 현재 MOVEIT_CONFIGS_PACKAGE_NAME 과 mock 전용 파일 경로에 하드코딩되어 있다. 다음을 인자화한다.

- description xacro 경로(포크 위치).
- ros2_control_hardware_type 매핑 값(이미 LaunchConfiguration 으로 주입됨 — 값 도메인만 gazebo/isaac 으로 확장).
- 나머지 SRDF/kinematics/joint_limits/planning_pipelines 는 백엔드 무관하게 공유(현행 유지).

8. 설정(Config) 전략

8.1 중복 제거: config_args_helper.py

rdfp_panda_mock / replay_panda_mock 에 복붙된 _declare_arguments() 와 _load_config() / _as_launch_str() 를 단일 helper 모듈로 추출한다. A/B 분리 후 argument 선언은 이 helper 한 곳에서만 이뤄진다.

8.2 백엔드별 YAML 분리

공통 앱 설정(rdfp_panda_mock.yaml)은 유지하되, 백엔드 고유 항목은 별도 키 또는 별도 파일로 둔다.

```
# config/rdp_panda_mock.yaml (앱 공통 - 기존 유지)
ros2_control: { hardware_type: mock_components }
camera: { enabled: true, id: 4, ... }
...
```

```
# config/backends/gazebo.yaml (신규)
world: empty.sdf
use_sim_time: true
camera: { enabled: false }      # 시물 센서가 대체
```

```
# config/backends/isaac.yaml (신규)
bridge_mode: ros2_control      # or topic_bridge
use_sim_time: true
camera: { enabled: false }
```

backend:=gazebo 일 때 bringup 이 backends/gazebo.yaml 을 머지하여 카메라 노드를 자동으로 끄는 식. (DSN 등 민감정보가 아니므로 평문 YAML 무방 — DB DSN 만 기존대로 env 참조 유지.)

8.3 use_sim_time 전파

Gazebo/Isaac 은 시물 클럭을 쓰므로 use_sim_time:=true 를 **Layer A 포함 모든 노드**에 전파해야 TF/타임스탬프가 어긋나지 않는다. mock 은 false. 이 값을 bringup 의 backend 선택에 따라 자동 결정하여 app launch 로 내려보낸다.

→ recorder/dataset 의 타임스탬프 세그멘테이션과 직결되므로 백엔드 추가 시 가장 주의할 항목.

9. 마이그레이션 단계 (점진적, 각 단계 독립 검증 가능)

- 원칙: 각 단계는 기존 mock 동작을 깨지 않은 채 끝나야 한다. 매 단계 후 `ros2 launch rdfp rdfp_bringup.launch.py` 가 현재와 동일하게 동작하는지 회귀 확인.
- **Phase 0 — 계약 명문화 (코드 변경 없음)**
본 문서의 § 2.3 / § 5 표를 docs/simulation/backend_contract.md 로 분리하고, 토픽/액션/프레임/QoS/관절이름을 동결. 이후 모든 백엔드의 수용 기준이 된다.
 - **Phase 1 — A/B 분리 리팩터링 (mock 만, 동작 동일)**
 1. config_args_helper.py 로 argument/YAML 로딩 중복 제거.
 2. app/rdfp_app.launch.py 로 Layer A 추출 (mode:=teleop|replay 인자로 기존 두 파일의 앱 노트셋 통합).
 3. backends/backend_mock.launch.py 로 ros2_control 스택 추출.
 4. bringup/rdfp_bringup.launch.py 가 둘을 include. 기존 rdfp_panda_mock.launch.py 등은 bringup 을 호출하는 thin 래퍼로 축소.
→ 산출물: 시뮬레이터는 아직 mock 뿐이지만, 구조가 백엔드 교체 가능 형태로 전환.
 - **Phase 2 — description 포크 + 물리 속성 (mock 회귀 유지)**
src/rdfp/description/ 에 panda xacro 포크, inertial/collision 보강, build_moveit_config() 일반화. mock 으로 회귀 검증(물리 속성은 mock 에 무해).
 - **Phase 3 — 기동 신호 추상화**
create_controller_startup_handlers() 를 "백엔드 ready 이벤트" 추상으로 일반화 (spawner-exit 기반 mock/gazebo, readiness-gate 기반 isaac 을 동일 인터페이스로).
 - **Phase 4 — backend_gazebo 구현**
Fortress 기동 + 스폰 + gz_ros2_control + 카메라 브리지 + use_sim_time 전파. backend:=gazebo 수용 기준(§ 5) 충족 검증.
 - **Phase 5 — backend_isaac 구현**
연결형 bridge + (필요시) trajectory adapter + readiness gate + USD↔URDF 정합성. backend:=isaac 수용 기준 충족 검증.

10. 리스크 및 주의사항

리스크	영향	완화
use_sim_time 누락/불일치	recorder/dataset 타임스탬프 오류	bringup 이 backend 별로 강제 주입(§ 8.3), 계약에 명시
외부 moveit_resources xacro 의존	Gazebo 하드웨어 분기 불가	description 포크(§ 7.1) — Phase 2 에서 선행
기동 신호가 hand_controller spawner 에 강결합	Isaac(spawner 없음) 적용 불가	ready 이벤트 추상화(§ 9 Phase 3)

리스크	영향	완화
launch sibling import / setup.py glob	디렉터리 이동 시 import·설치 실패	sys.path 패턴 유지 + data_files glob 을 하위 디렉터리 포함하도록 확장(§ 4)
controller 이름/관절 불일치	Movelt 실행 실패	계약(§ 5.2)으로 동결, 백엔드 수용 기준에 포함
Isaac USD↔URDF 드리프트	TF/충돌/관절 매핑 깨짐	URDF 를 단일 진실원본으로 두고 USD 를 거기서 생성/검증
카메라 토픽 이름 분기	dataset 파이프라인 깨짐	토픽 이름을 계약으로 고정, 소스만 교체(§ 5.4)

11. 요약

- 교체 대상은 **RViz 가 아니라 하드웨어 백엔드**다. rdfp 는 이미 ros2_control
 - 토픽 계약 위에 있어 추상화 경계가 존재한다.
- 핵심 작업은 **launch** 를 **app(A)/backend(B)** 로 분리하고, 모든 백엔드가 동일한 ****계약(§ 5)****을 만족하게 만드는 것이다.
- mock → Gazebo → Isaac 은 백엔드 launch 파일 + URDF 하드웨어 분기만 추가하면 되고, **앱·Movelt·dataset 계층은 영구 불변**이다.
- 선행 가치가 가장 큰 작업은 **Phase 1(A/B 분리 + 중복 제거)** 과 **Phase 2(description 포크 + 물리 속성)** 이며, 이 둘은 mock 동작을 깨지 않고 지금 바로 진행 가능하다.